BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

LESSON 057 · Lesson 057 · C# Data Structures Mini-project: Library Loans

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What does a constructor initialise?

Q2. What does Queue<T>.Dequeue do?

Q3. Why choose a unique ID for a record?

Q4. 017

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

model — Representation of a real-world concept.

constructor — Initialises new object.

Queue<T> — Generic FIFO collection.

validation — Checks input meets rules.

openWrite — Open an OCR pseudocode file for writing, replacing previous contents.

openRead — Open an OCR pseudocode file for reading stored lines.

New learning

Understand — Class Loan models a single loan request, containing a student and book title initialised in its constructor.

Model — Queue<Loan> stores objects in requested service order. Enqueue adds a new request and Dequeue retrieves oldest.

Evaluate — For a more complete system, use a Dictionary keyed by unique loan ID, validation and tests; do not treat a name as necessarily unique. A queue exists only while the program keeps it in memory; a text file can persist loan records. In OCR examination pseudocode, openWrite plus writeLine writes a file (overwriting existing contents), while openRead plus readLine retrieves lines. Always close() the file.

Worked example

Trace two loan objects in a queue: Ida first, Tom second. Which is processed first?

1. Explain — Queue is FIFO, so the first enqueued request is processed first.

2. Why a file? — Unlike the queue, the text file can hold a record across program runs. openWrite creates a write stream and replaces any existing contents of the target file; it is not an append operation.

3. Trace the file contents — First writeLine stores Ada,Holes , then the second stores Ben,Dune . They become two separate text lines. This is a simple illustrative format, not a robust CSV parser.

4. Explain the effect — f.close() finishes the write operation and releases the file. Do not confuse storing records in a queue or dictionary with saving them permanently.

We Do · Practise with guidance

A different library uses Dictionary<int,Loan> keyed by ID, entries 10 and 12. Which ID does lookup 12 retrieve?

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

```
loansById[12] returns the Loan stored under ID 12, regardless of insertion
  order.
```

Explain

A dictionary retrieves by key; it does not enforce FIFO service order.

Read loans.txt , containing Ada,Holes and Ben,Dune on separate lines.

```
// OCR H446 examination pseudocode
f = openRead("loans.txt")
while NOT f.endOfFile()
    print(f.readLine())
endwhile
f.close()
```

Predict what is printed

Identify the role of endOfFile() and the effect of each readLine() call.

Follow two reads

The first call prints Ada,Holes ; the second prints Ben,Dune . The file position advances after each read.

Stop safely

Once no unread lines remain, endOfFile() is true, the loop stops, and f.close() releases the reader. This example reads text; it does not reconstruct Loan objects automatically.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Explain two reasons Loan is a class rather than separate parallel arrays for pupil and book. [4]

Question 2

Write C# creating a Queue<Loan>, enqueue two instances, then safely dequeue one using Count. [8]

Question 3

Outline how to prevent duplicate loan IDs and reject empty student names; then name the OCR file operations for writing one saved loan and reading a line back. [8]

Exit Ticket · Answer before checking

A pupil's name is always guaranteed unique, so it is a perfect database key.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. New object's fields/properties.

Q2. Removes/returns oldest enqueued item.

Q3. Names/labels can repeat; ID enables unambiguous lookup.

Q4. A C# int[,] 3×2 array has how many values and last index pair?

You Do pages · Independent questions

Q1. Related fields bundled [2]; object methods/encapsulation/maintainability [2].

Q2. Queue<Loan> q=new(); [2], two q.Enqueue(new Loan(...)) [4], if(q.Count>0) [1], Loan next=q.Dequeue() [1].

Q3. Dictionary<int,Loan> keyed by unique ID, reject duplicate keys [2]; validate a nonempty student name [2]; openWrite + writeLine (+ close) to create/write saved text [2]; openRead + readLine (+ close) to retrieve text [2].

Exit Ticket · Model answer

False. Use a stable unique identifier.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: